Nandhika . M
B.Tech CSE (IoT) - A
2301110 2053

CS3002 - CRYPTOGRAPHY CONCEPTS

CIA

HASH GENERATED GRONSFELD CIPHER ALGORITHM :

ALGORITHM : Hash_Gronsfeld_cipher

INPUT : Plaintext, P (string), key-length, k (integer)

OUTPUT : ciphertext, C (string), numeric_key K (list of digits)

ALGORITHM HASH_GRONSFELD_CIPHER (P, k) :

1. HASH GENERATION (djb2)

```
hash_value ← 5381
FOR each character c in P :
    hash_value ← ((hash_value x 33) + ord (c))
H ← hash_value
```

2. KEY EXTRACTION

```
digits ← H mod (10^k)
key_str ← decimal string of digits, padded with leading
          zeroes to length k.
K ← convert key_str to array of integers [d1, d2, ..., dk]
```

3. ENCRYPTION

```
C ← empty string
key_index ← 0
FOR each character ch in P :
    IF ch is alphabetic :
        base ← 'A' if uppercase else 'a'
        pos ← ord (ch) - ord (base)
        shift ← K [key_index mod k]
        new_pos ← (pos + shift) mod 26
```

```
            new_char ← chr(ord(base) + new_pos)
            APPEND new_char to C
            key_index ← key_index + 1

        ELSE :
            APPEND ch to C

4. RETURN (C, K)

5. DECRYPTION
    DECRYPT(ciphertext C, numeric_key K) :
        D ← empty_string
        key_index ← 0
        FOR each character ch in C :
            IF ch is alphabetic :
                base ← 'A' if uppercase else 'a'
                pos ← ord(ch) - ord(base)
                shift ← K [key_index mod k]
                new_pos ← (pos - shift + 26) mod 26
                new_char ← chr(ord(base) + new_pos)
                APPEND new_char to D
                key_index ← key_index + 1
            ELSE :
                Append ch to D

    RETURN D.
```

djb2 HASH FUNCTION :

- Converts any string into one unique large number.
- Working:
  1. Start with magic number : 5381
  2. For each letter in message :
     a. Multiply current hash by 33.
     b. Add the letter's position number (A = 65, B = 66)...
     c. Repeat for next letter
  3. Final result = one big hash number.

WORKFLOW :

Plaintext P → [djb2 Hash] → H → [mod 10^k + pad]

↓

Key K ← ciphertext / Plaintext

⇓

Use the same K for encryption and decryption

Time complexity : O(n) where n - length of plaintext

Space complexity : O(k) where k - fixed key length

Example :

- Input : "HELLO WORLD" (Plaintext)
- key length = 8
- Execution :
  1. DJB2 Hash ("HELLO WORLD") = 4253865281374928
  2. Key Extraction :
     4253865281374928 mod 10^8 = 371374928
     Key = [3, 7, 1, 3, 7, 4, 9, 2, 8]

3. Encryption:

H+3 = K, E+7 = L, L+1 = M, L+3 = O, O+7 = V

Space → Space

W+4 = A, O+9 = X, R+2 = T, L+8 = T, D+3 = G

Ciphertext = "KLMOV AXTT"

4. Saved Key = [3, 7, 1, 3, 7, 4, 9, 2, 8]

5. Decryption:

K-3 = H, L-7 = E, M-1 = L, O-3 = L, V-7 = O

Space → Space

A-4 = W, X-9 = O, T-2 = R, T-8 = L, G-3 = D

Plaintext: "HELLO WORLD".

~~ADVANTAGES~~

SETBACKS & MITIGATIONS:

- Hash collisions possible → mitigated by longer keys (20+ digits)
- Not cryptographically secure → use SHA 256 for production
- Key must be transmitted → Send key along with ciphertext.